Space Complexity

**Example 1:**

```
Algorithm abc(a,b,c)
{
return a+b++*c+(a+b-c)/(a+b) +4.0;
}
```

→ The Space needed by each of these algorithms is seen to be the sum of the following component.

1. A **fixed part** that is independent of the characteristics (eg: number, size) of the inputs and outputs.

The part typically includes the instruction space (ie. Space for the code), space for simple variable and fixed-size component variables (also called aggregate) space for constants, and so on.

A **variable part** that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that is depends on instance characteristics), and the recursion stack space.

The space requirement s(p) of any algorithm p may therefore be written as,

**S(P) = c+ Sp(Instance characteristics)**

Where 'c' is a constant.

**Example 2:**

```
Algorithm sum(a,n)
{
    s=0.0;
    for I=1 to n do
    s= s+a[I];
    return s;
}
```

The problem instances for this algorithm are characterized by n,the number of elements to be summed. The space needed d by 'n' is one word, since it is of type integer.

The space needed by 'a'a is the space needed by variables of type array of floating point numbers.

This is at least 'n' words, since 'a' must be large enough to hold the 'n' elements to be summed.

So, we obtain Ssum(n)>=(n+s)

[ n for a[],one each for n,I a& s]